Manipulación del Dispositivo

Persistencia e integración

Backend-as-a-Service

Objetivos de la clase

Al finalizar esta clase, el alumno será capaz de:

- Comprender los diferentes niveles de persistencia de datos.
- Usar AsyncStorage para gestionar datos locales de forma segura y controlada.
- Implementar autenticación y gestión de datos con Firebase.
- Sincronizar de forma básica el estado local de la app con una base de datos remota.
- Evaluar opciones gratuitas de AWS para servicios de backend integrables con React Native.

1. ¿Qué es la persistencia de datos?

La persistencia se refiere a la capacidad de un sistema para mantener datos más allá del ciclo de vida de la aplicación. En el caso de apps móviles, esto significa que los datos no se pierdan al cerrar la app, apagar el teléfono o incluso reinstalar.

1.1. ¿Por qué es importante?

- Permite retener configuraciones, sesiones y preferencias.
- Mejora la experiencia del usuario (UX).
- Reduce llamadas innecesarias al servidor.
- Facilita el funcionamiento offline (modo desconectado).

1.2a Tipos de persistencia

Tipo	Alcance	Ejemplo
🧠 Memoria	Volátil, se pierde al cerrar app	`useState`, `useContext`
📦 Local	Persistente, dispositivo	`AsyncStorage`, SQLite
☁️ Remota	Persistente, servidor o nube	Firebase, AWS, MongoDB Atlas

1.2b ¿Cuál conviene usar según el caso?

Memoria (useState, useContext)

- Ideal para datos temporales que solo importan mientras la app está abierta (ej: estado de un formulario, valor de un switch, navegación interna).
- No sirve si necesitás que los datos sigan estando al cerrar o reiniciar la app.

Local (AsyncStorage, SQLite)

- Útil para datos persistentes del usuario en el dispositivo: sesión iniciada, configuraciones, progreso de un juego, favoritos, historial offline.
- Funciona sin conexión a internet.
- Solo accesible desde el dispositivo, no se comparte entre varios usuarios/equipos.

Remota (Firebase, AWS, MongoDB Atlas)

- Necesaria para datos compartidos entre varios usuarios o dispositivos: cuentas, chats, compras, sincronización en la nube.

- Permite backups y acceso desde cualquier lado.
- Requiere conexión a internet y configuración de backend/servicio.

2. Almacenamiento local con AsyncStorage

2.1 ¿Qué es AsyncStorage?

AsyncStorage es un sistema simple de almacenamiento de pares clave-valor, provisto por React Native y Expo mediante una biblioteca externa, que actúa como un wrapper persistente sobre bases nativas (SharedPreferences en Android, UserDefaults en iOS).
Ideal para:

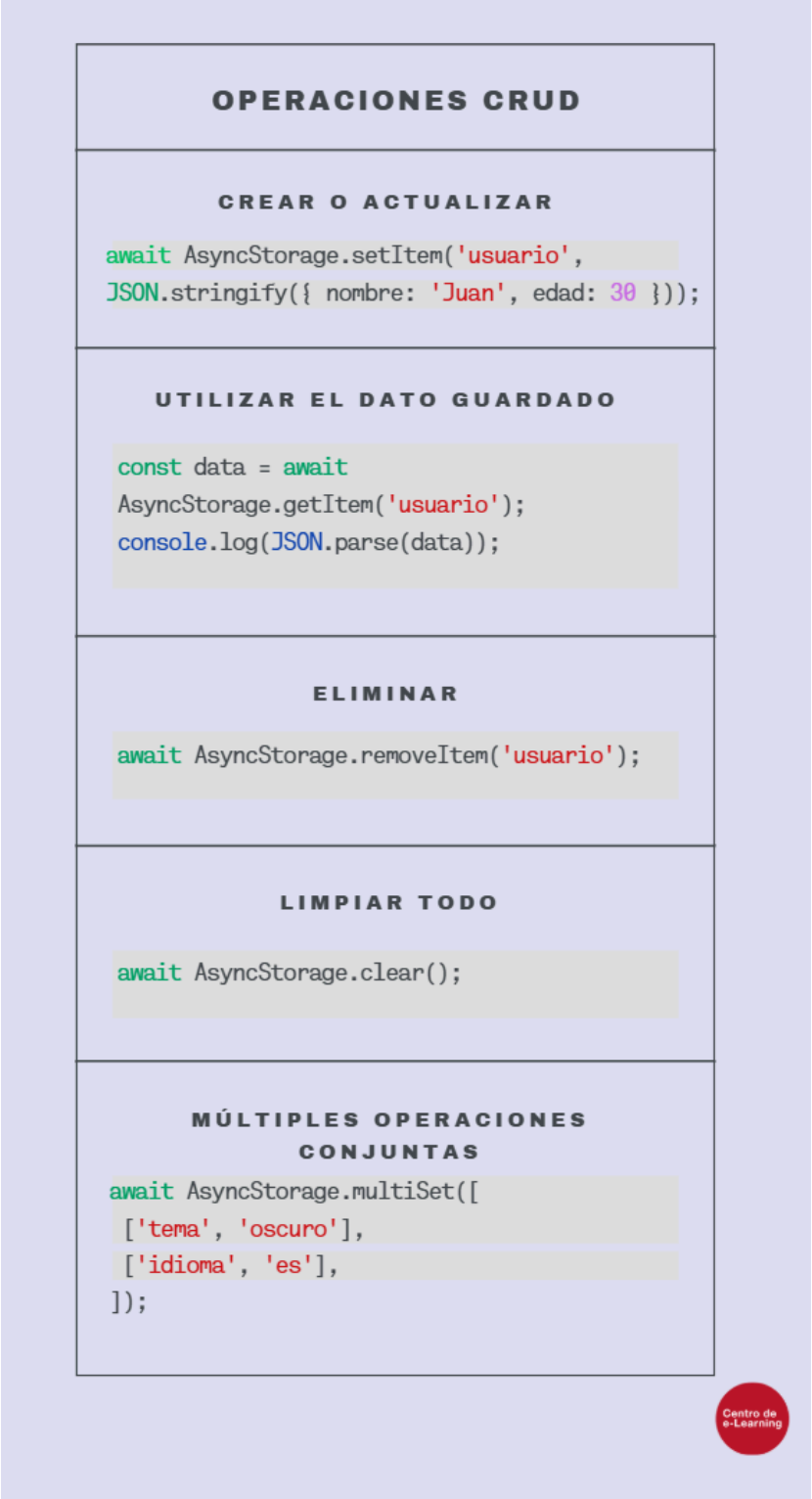
- Tokens de sesión
- Configuraciones del usuario
- Datos temporales

2.2 Instalación y setup

```
npx expo install @react-native-async-storage/async-storage
```

```
import AsyncStorage from
'@react-native-async-storage/async-storage';
```

2.3 Operaciones CRUD



2.4 Buenas prácticas

- Usar JSON.stringify() y JSON.parse() para objetos complejos.
- Agrupar claves con prefijos: @app:user, @app:config.
- Usar try/catch en todos los accesos para evitar crashes.
- No almacenar información sensible (como contraseñas) sin cifrado.

3. Firebase como Backend-as-a-Service

3.1 ¿Qué es Firebase?

Es una plataforma mantenida por Google que centraliza múltiples servicios backend (autenticación, base de datos, storage, notificaciones, hosting, analytics, etc.). Firebase permite lanzar MVPs sin preocuparse por servidores, escalabilidad o configuración inicial.

3.2 Ventajas principales

Ventaja	Detalle
✅ Escalabilidad	Arquitectura serverless lista para crecer con tu app.
✅ Tiempo real	Cambios sincronizados automáticamente entre todos los dispositivos.
✅ SDK multiplataforma	Web, Android, iOS, React Native, Flutter.
✅ Plan gratuito	Suficiente para MVPs, demos y proyectos pequeños.

4. Firebase Authentication

```
firebaseConfig.js
├── Auth (correo, Google, Facebook...)
├── Firestore (colecciones y documentos)
```

```
├── Storage (archivos)
├── Rules (seguridad y control de acceso)
```

4.1 Registro con email y contraseña

```
import { getAuth, createUserWithEmailAndPassword } from
'firebase/auth';

const auth = getAuth();
createUserWithEmailAndPassword(auth, email, password);
```

4.2 Login

```
import { signInWithEmailAndPassword } from 'firebase/auth';
signInWithEmailAndPassword(auth, email, password);
```

4.3 Logout

```
import { signOut } from 'firebase/auth';
await signOut(auth);
```

4.4 Observador global de sesión

```
import { onAuthStateChanged } from 'firebase/auth';

useEffect(() => {
  const unsub = onAuthStateChanged(auth, usuario =>
setUser(usuario));
  return unsub;
}, []);
```

4.5 Alternativas

Método	SDK / integración
Login con Google	`expo-auth-session`, OAuth
Login anónimo	Firebase Built-in
Login con Facebook	Facebook SDK + Firebase

5. Firestore (Base de datos NoSQL)

5.1 ¿Qué es Firestore?

Es una base de datos documental y en tiempo real, donde los datos se organizan en:

- Colecciones → Documentos → Campos (clave-valor)

Ideal para apps con sincronización inmediata, chats, tareas, perfiles, etc.

5.2 Escritura de datos

```
import { addDoc, collection } from 'firebase/firestore';

await addDoc(collection(db, 'tareas'), {
  titulo: 'Crear clase 9',
  completado: false,
});
```

5.3 Lectura en tiempo real

```
import { onSnapshot } from 'firebase/firestore';

useEffect(() => {
  const unsub = onSnapshot(collection(db, 'tareas'), snapshot =>
{
    const docs = snapshot.docs.map(doc => ({ id: doc.id,
...doc.data() }));
    setTareas(docs);
  });
  return unsub;
}, []);
```

5.4 Reglas de seguridad básicas

```
match /tareas/{docId} {
  allow read, write: if request.auth != null;
}
```

6. Sincronización entre local y remoto

6.1 Concepto

Consiste en guardar datos localmente con AsyncStorage cuando no hay conexión, y luego enviarlos al servidor (Firestore) cuando se restablece la red.

6.2 Requisitos

- Manejo de conectividad con @react-native-community/netinfo.
- Almacenamiento local temporal.
- Lógica de reintento.

```
import NetInfo from '@react-native-community/netinfo';

NetInfo.addEventListener(state => {
  if (state.isConnected) {
```

```
    sincronizarPendientes();
  }
});
```

6.3 Estrategia general

- Guardar acciones pendientes localmente.
- Verificar conexión cada X segundos o con evento.
- Enviar y limpiar al confirmar sincronización.
- Feedback visual al usuario (“Sin conexión” / “Datos sincronizados”).

7. Alternativa: AWS Backend-as-a-Service (BaaS)

7.1 ¿Qué ofrece AWS?

AWS tiene una gran cantidad de servicios gratuitos para el primer año. Algunos de ellos permiten desarrollar un backend completo sin costos para proyectos chicos o MVPs.

7.2 Servicios gratuitos útiles

Servicio	Funcionalidad	Límite gratuito mensual
AWS Cognito	Autenticación y gestión de usuarios	50.000 MAUs (usuarios activos únicos)
DynamoDB	Base de datos NoSQL	25 GB + 200M requests
S3	Almacenamiento de archivos	5 GB
AppSync	Sincronización GraphQL en tiempo real	250.000 solicitudes
Lambda	Backend sin servidor (serverless)	1 millón de invocaciones

7.3 AWS Amplify + React Native

Amplify es el framework oficial de AWS para apps móviles. Aunque no es compatible directamente con Expo sin ejecutar **eject**, se puede usar en apps "bare".

Requiere CLI (npm install -g @aws-amplify/cli), una cuenta de AWS y configuraciones avanzadas.

7.4 Comparativa Firebase vs AWS (plan gratuito)

Característica	Firebase	AWS (Amplify / BaaS)
Facilidad de inicio	Muy alta	Media (requiere CLI / IAM)
Escalabilidad	Alta	Muy alta
Autenticación	Integrada	Cognito
Base de datos	Firestore (NoSQL)	DynamoDB (NoSQL)
Almacenamiento	Storage	S3
Sincronización offline	Sí	Sí (con AppSync)



Bibliografía utilizada y sugerida

Expo Team. Using AsyncStorage.

https://docs.expo.dev/versions/latest/sdk/async-storage/

Actualizado en Julio 2024.

Firebase Docs. Firebase Authentication.

https://firebase.google.com/docs/auth

Actualizado en Julio 2024.

Firebase Docs. Firestore Database.

https://firebase.google.com/docs/firestore

Actualizado en Julio 2024.

AWS Docs. Free Tier Services.

https://aws.amazon.com/free/

Actualizado en Junio 2024.

AWS Amplify. React Native Guide.

https://docs.amplify.aws/start/q/integration/react-native/

Actualizado en Julio 2024.